

OpenSlot

Testing Summary Report

Benjamin Okojie, 169075607
Dharmik Patel, 169033371
Akif Rahman, 210290530

April 5, 2026

CP476: Internet Computing - Winter 2026
Dr. Mustafa Daraghmeh

1. Overview

This report summarizes the testing completed for the Milestone 03 release of OpenSlot.

The goal of testing was to validate that the backend API behaves correctly across all implemented features using automated testing. The testing focused on:

- authentication and profile management
- course retrieval
- slot management
- appointment lifecycle
- schedule viewing
- validation and business rules

2. Test Environment

- Back-end: Node.js + Express
- Framework: Vitest
- HTTP Testing: Supertest
- Data: In-memory demo data
- Execution Date: April 5, 2026

3. Test Strategy

3.1 Functional Testing

Endpoints were tested individually to ensure correctness.

3.2 Automated Testing

The full API was tested through automated HTTP requests.

4. Test Plan

4.1 Feature Coverage

Authentication: 11 test cases

Courses: 4 test cases

Slots: 21 test cases

Appointments: 25 test cases

Schedule: 7 test cases

Total: 68 test cases

4.2 Detailed Test Cases

Authentication

- Login valid student → 200
- Login valid professor → 200
- Unknown email → 401
- Missing email → 401
- Case insensitive login → Success
- Logout → ok:true
- Update profile valid → Updated
- Update non-existent user → 404
- Invalid userId → 400
- Blank name → 400
- Duplicate email → 409

Courses

- Fetch courses → Success
- Count = 5 → Valid
- Schema valid → Valid
- Codes exist → Valid

Slots

- Fetch slots → Success
- Default filtering → Valid
- Include booked → Valid
- Filter course → Valid
- Filter professor → Valid
- Filter status → Valid
- Enriched data → Present
- Sorted order → Correct
- Get by ID → Success
- Invalid ID → 404
- Enriched fields → Present
- Create valid slot → 201

- Default draft → Correct
- Missing professor → 400
- Missing start → 400
- Missing location → 400
- Draft → posted → 200
- Cancel slot → 200
- Cancel booked clears → Valid
- Invalid status → 400
- Non-existent slot → 404

Appointments

- Book slot → 201
- Double book → 409
- Missing slot_id → 400
- Invalid student → 404
- Invalid slot → 404
- Draft slot booking → 409
- Get bookings → Success
- Enriched data → Present
- Invalid ID → 400
- Booking exists → Valid
- Past → completed → Valid
- Get details → Success
- Student info → Present
- Invalid ID → 404
- Invalid format → 400
- Cancel booking → Success
- Cancel again → 409
- Cancel past → 409
- Cancel invalid → 404
- Reschedule → Success
- Same slot → 400
- Booked slot → 409
- Missing new slot → 400
- Invalid slot → 404
- Cancelled reschedule → 409

Schedule

- Fetch schedule → Success

- Week range → Valid
- Day view → Valid
- Enriched data → Present
- Professor filter → Valid
- Invalid date → 400
- Empty case → Valid

5. Results

- Total Tests: 68
- Passed: 68
- Failed: 0

5.2 Summary of Results

- All core Milestone 03 API workflows covered by the automated test suite passed successfully.
- Endpoint behavior is consistent across authentication, slots, appointments, and scheduling features.
- Business rules such as double-booking prevention, slot status transitions, and rescheduling logic are enforced correctly.
- Validation checks correctly reject invalid inputs, missing fields, and unauthorized operations.
- Data relationships between slots, appointments, courses, and users are returned correctly through enriched responses.
- No blocking defects were identified in the tested API functionality.

6. Automated Tests

6.1 How to Run

```
cd server  
npm install  
npm run verify
```

6.2 What the Automated Tests Do

The automated test suite performs end-to-end verification of the API by simulating real client interactions.

The test suite:

- initializes the Express server in a controlled test environment
- sends HTTP requests to all major API endpoints
- logs in as both student and professor users
- verifies authentication behavior and role-based logic
- retrieves course data and validates schema correctness
- fetches, filters, and validates slot data
- creates new slots and verifies required field validation
- updates slot statuses (draft → posted → cancelled)
- books available slots and ensures double-booking is prevented
- retrieves student bookings and appointment details
- validates enriched response data (course, professor, and student information)
- cancels appointments and verifies slot availability updates
- reschedules appointments and confirms correct slot transitions
- validates professor schedule endpoints (day and week views)
- checks error handling for invalid IDs, missing inputs, and invalid formats

7. Known Issues / Limitations

- Uses in-memory demo data; data resets on server restart
- Authentication tokens are not enforced via middleware
- No real authentication security (password hashing, sessions, etc.)
- Some business rules rely on controlled demo data
- No automated frontend testing included
- Input validation is basic and not strict

8. Conclusion

Based on the completed testing, OpenSlot satisfies the main Milestone 02 expectations for a full-stack application:

- supports full end-to-end scheduling workflow
- implements CRUD operations for slots and appointments
- enforces business rules and validation logic
- includes strong automated test coverage
- behaves consistently across normal and edge cases

Overall, the system is stable and ready for submission, with remaining issues being non-blocking improvements for future milestones.